

第31章 ADC—电压采集

本章参考资料：《STM32F4xx 中文参考手册》ADC 章节。

学习本章时，配合《STM32F4xx 中文参考手册》ADC 章节一起阅读，效果会更佳，特别是涉及到寄存器说明的部分。

31.1 ADC 简介

STM32F407ZGT6 有 3 个 ADC，每个 ADC 有 12 位、10 位、8 位和 6 位可选，每个 ADC 有 16 个外部通道。另外还有两个内部 ADC 源和 V_{BAT} 通道挂在 ADC1 上。ADC 具有独立模式、双重模式和三重模式，对于不同 AD 转换要求几乎都有合适的模式可选。ADC 功能非常强大，具体的我们在功能框图中分析每个部分的功能。

31.2 ADC 功能框图剖析

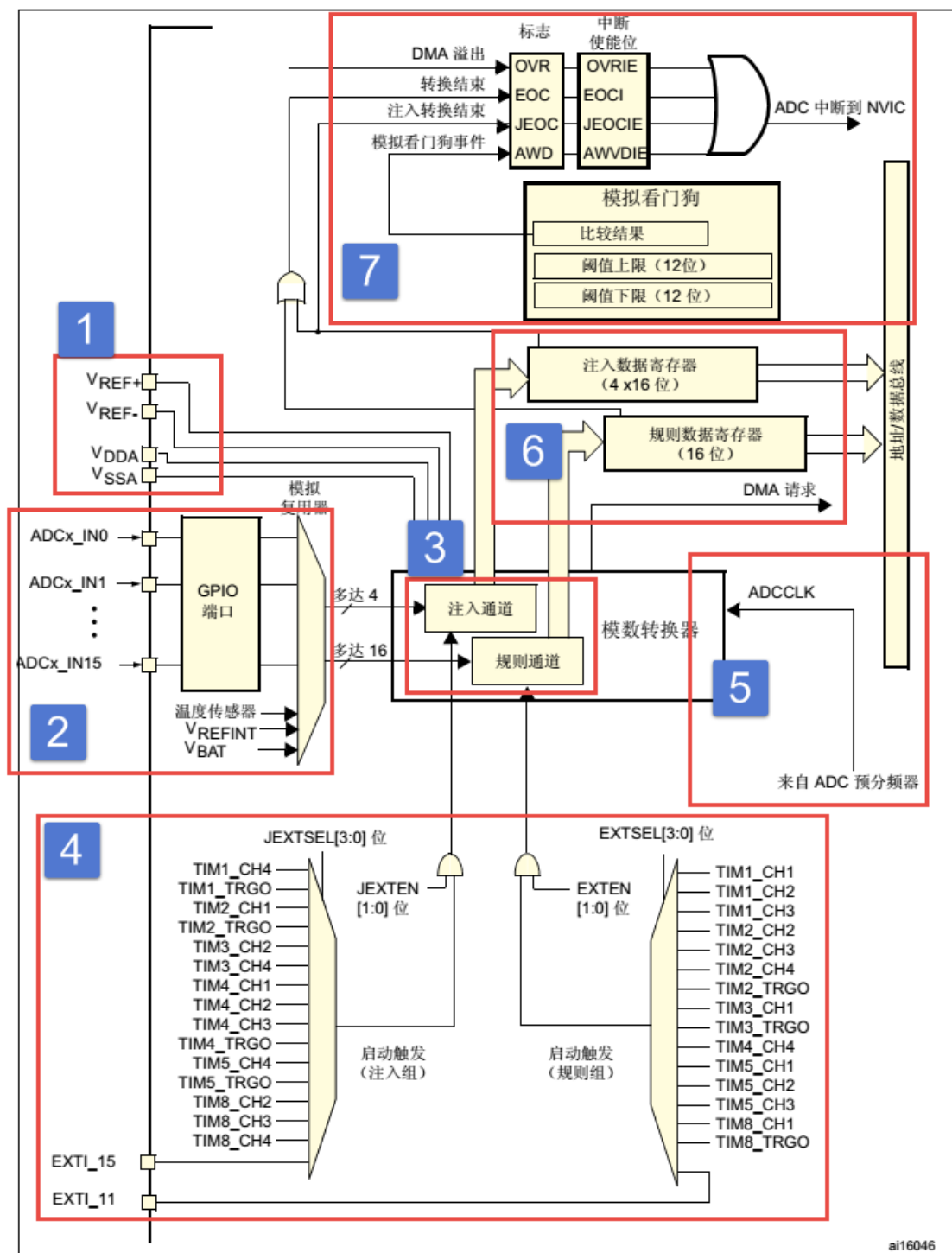


图 31-1 单个 ADC 功能框图

掌握了 ADC 的功能框图，就可以对 ADC 有一个整体的把握，在编程的时候可以做到了然如胸，不会一知半解。框图讲解采用从左到右的方式，跟 ADC 采集数据，转换数据，传输数据的方向大概一致。

1. ①电压输入范围

ADC 输入范围为： $V_{REF-} \leq V_{IN} \leq V_{REF+}$ 。由 V_{REF-} 、 V_{REF+} 、 V_{DDA} 、 V_{SSA} 、这四个外部引脚决定。

我们在设计原理图的时候一般把 V_{SSA} 和 V_{REF-} 接地，把 V_{REF+} 和 V_{DDA} 接 3V3，得到 ADC 的输入电压范围为：0~3.3V。

如果我们想让输入的电压范围变宽，去到可以测试负电压或者更高的正电压，我们可以在外部加一个电压调理电路，把需要转换的电压抬升或者降压到 0~3.3V，这样 ADC 就可以测量了。

2. ②输入通道

我们确定好 ADC 输入电压之后，那么电压怎么输入到 ADC？这里我们引入通道的概念，STM32 的 ADC 多达 19 个通道，其中外部的 16 个通道就是框图中的 $ADCx_IN0$ 、 $ADCx_IN1 \dots ADCx_IN5$ 。这 16 个通道对应着不同的 IO 口，具体是哪一个 IO 口可以从手册查询到。其中 ADC1/2/3 还有内部通道：ADC1 的通道 $ADC1_IN16$ 连接到内部的 VSS，通道 $ADC1_IN17$ 连接到了内部参考电压 V_{REFINT} 连接，通道 $ADC1_IN18$ 连接到了芯片内部的温度传感器或者备用电源 V_{BAT} 。ADC2 和 ADC3 的通道 16、17、18 全部连接到了内部的 VSS。

STM32F407ZGT6 ADC IO 分配					
ADC1	IO	ADC2	IO	ADC3	IO
通道0	PA0	通道0	PA0	通道0	PA0
通道1	PA1	通道1	PA1	通道1	PA1
通道2	PA2	通道2	PA2	通道2	PA2
通道3	PA3	通道3	PA3	通道3	PA3
通道4	PA4	通道4	PA4	通道4	PF6
通道5	PA5	通道5	PA5	通道5	PF7
通道6	PA6	通道6	PA6	通道6	PF8
通道7	PA7	通道7	PA7	通道7	PF9
通道8	PB0	通道8	PB0	通道8	PF10
通道9	PB1	通道9	PB1	通道9	PF3
通道10	PC0	通道10	PC0	通道10	PC0
通道11	PC1	通道11	PC1	通道11	PC1
通道12	PC2	通道12	PC2	通道12	PC2
通道13	PC3	通道13	PC3	通道13	PC3
通道14	PC4	通道14	PC4	通道14	PF4
通道15	PC5	通道15	PC5	通道15	PF5
通道16	连接内部温度传感器	通道16	连接内部VSS	通道16	连接内部VSS
通道17	连接内部Vrefint	通道17	连接内部VSS	通道17	连接内部VSS

图 31-2 STM32F407ZGT6 ADC 通道

外部的 16 个通道在转换的时候又分为规则通道和注入通道，其中规则通道最多有 16 路，注入通道最多有 4 路。那这两个通道有什么区别？在什么时候使用？

规则通道

规则通道：顾名思义，规则通道就是很规矩的意思，我们平时一般使用的就是这个通道，或者说我们用到都是这个通道，没有什么特别要注意的可讲。

注入通道

注入，可以理解为插入，插队的意思，是一种不安分的通道。它是一种在规则通道转换的时候强行插入要转换的一种。如果在规则通道转换过程中，有注入通道插队，那么就要先转换完注入通道，等注入通道转换完成后，再回到规则通道的转换流程。这点跟中断程序很像，都是不安分的主。所以，注入通道只有在规则通道存在时才会出现。

3. ③转换顺序

规则序列

规则序列寄存器有 3 个，分别为 SQR3、SQR2、SQR1。SQR3 控制着规则序列中的第一个到第六个转换，对应的位为：SQ1[4:0]~SQ6[4:0]，第一次转换的是位 4:0 SQ1[4:0]，如果通道 16 想第一次转换，那么在 SQ1[4:0]写 16 即可。SQR2 控制着规则序列中的第 7 到第 12 个转换，对应的位为：SQ7[4:0]~SQ12[4:0]，如果通道 1 想第 8 个转换，则 SQ8[4:0]写 1 即可。SQR1 控制着规则序列中的第 13 到第 16 个转换，对应位为：SQ13[4:0]~SQ16[4:0]，如果通道 6 想第 10 个转换，则 SQ10[4:0]写 6 即可。具体使用多少个通道，由 SQR1 的位 L[3:0]决定，最多 16 个通道。

规则序列寄存器 SQR _x , x (1, 2, 3)			
寄存器	寄存器位	功能	取值
SQR3	SQ1[4:0]	设置第1个转换的通道	通道1~16
	SQ2[4:0]	设置第2个转换的通道	通道1~16
	SQ3[4:0]	设置第3个转换的通道	通道1~16
	SQ4[4:0]	设置第4个转换的通道	通道1~16
	SQ5[4:0]	设置第5个转换的通道	通道1~16
	SQ6[4:0]	设置第6个转换的通道	通道1~16
SQR2	SQ7[4:0]	设置第7个转换的通道	通道1~16
	SQ8[4:0]	设置第8个转换的通道	通道1~16
	SQ9[4:0]	设置第9个转换的通道	通道1~16
	SQ10[4:0]	设置第10个转换的通道	通道1~16
	SQ11[4:0]	设置第11个转换的通道	通道1~16
	SQ12[4:0]	设置第12个转换的通道	通道1~16
SQR1	SQ13[4:0]	设置第13个转换的通道	通道1~16
	SQ14[4:0]	设置第14个转换的通道	通道1~16
	SQ15[4:0]	设置第15个转换的通道	通道1~16
	SQ16[4:0]	设置第16个转换的通道	通道1~16
	SQL[3:0]	需要转换多少个通道	1~16

图 31-3 规则序列寄存器

注入序列

注入序列寄存器 JSQR 只有一个，最多支持 4 个通道，具体多少个由 JSQR 的 JL[2:0] 决定。如果 JL 的值小于 4 的话，则 JSQR 跟 SQR 决定转换顺序的设置不一样，第一次转换的不是 JSQR1[4:0]，而是 JCQR_x[4:0]，x = (4-JL)，跟 SQR 刚好相反。如果 JL=00 (1

个转换)，那么转换的顺序是从 JSQR4[4:0]开始，而不是从 JSQR1[4:0]开始，这个要注意，编程的时候不要搞错。当 JL 等于 4 时，跟 SQR 一样。

注入序列寄存器 JSQR			
寄存器	寄存器位	功能	取值
JSQR	JSQ1[4:0]	设置第1个转换的通道	通道1~4
	JSQ2[4:0]	设置第2个转换的通道	通道1~4
	JSQ3[4:0]	设置第3个转换的通道	通道1~4
	JSQ4[4:0]	设置第4个转换的通道	通道1~4
	JL[1:0]	需要转换多少个通道	1~4

图 31-4 注入序列寄存器

4. ④触发源

通道选好了，转换的顺序也设置好了，那接下来就该开始转换了。ADC 转换可以由 ADC 控制寄存器 2: ADC_CR2 的 ADON 这个位来控制，写 1 的时候开始转换，写 0 的时候停止转换，这个是最简单也是最好理解的开启 ADC 转换的控制方式，理解起来没啥技术含量。

除了这种庶民式的控制方法，ADC 还支持外部事件触发转换，这个触发包括内部定时器触发和外部 IO 触发。触发源有很多，具体选择哪一种触发源，由 ADC 控制寄存器 2:ADC_CR2 的 EXTSEL[2:0]和 JEXTSEL[2:0]位来控制。EXTSEL[2:0]用于选择规则通道的触发源，JEXTSEL[2:0]用于选择注入通道的触发源。选定好触发源之后，触发源是否要激活，则由 ADC 控制寄存器 2:ADC_CR2 的 EXTTRIG 和 JEXTTRIG 这两位来激活。

如果使能了外部触发事件，我们还可以通过设置 ADC 控制寄存器 2:ADC_CR2 的 EXTEN[1:0]和 JEXTEN[1:0]来控制触发极性，可以有 4 种状态，分别是：禁止触发检测、上升沿检测、下降沿检测以及上升沿和下降沿均检测。

5. ⑤转换时间

ADC 时钟

ADC 输入时钟 ADC_CLK 由 PCLK2 经过分频产生，最大值是 36MHz，典型值为 30MHz，分频因子由 ADC 通用控制寄存器 ADC_CCR 的 ADCPRE[1:0]设置，可设置的分频系数有 2、4、6 和 8，注意这里没有 1 分频。对于 STM32F429IGT6 我们一般设置 $PCLK2=HCLK/2=84MHz$ 。所以程序一般使用 4 分频或者 6 分频。

采样时间

ADC 需要若干个 ADC_CLK 周期完成对输入的电压进行采样，采样的周期数可通过 ADC 采样时间寄存器 ADC_SMPR1 和 ADC_SMPR2 中的 SMP[2:0]位设置，ADC_SMPR2 控制的是通道 0~9，ADC_SMPR1 控制的是通道 10~17。每个通道可以分别用不同的时间采样。其中采样周期最小是 3 个，即如果我们要达到最快的采样，那么应该设置采样周期为 3 个周期，这里说的周期就是 $1/ADC_CLK$ 。

ADC 的总转换时间跟 ADC 的输入时钟和采样时间有关，公式为：

$$T_{\text{conv}} = \text{采样时间} + 12 \text{ 个周期}$$

当 $\text{ADCCLK} = 30\text{MHz}$ ，即 PCLK2 为 60MHz ，ADC 时钟为 2 分频，采样时间设置为 3 个周期，那么总的转换时为： $T_{\text{conv}} = 3 + 12 = 15 \text{ 个周期} = 0.5\mu\text{s}$ 。

一般我们设置 $\text{PCLK2} = 84\text{MHz}$ ，经过 ADC 预分频器能分频到最大的时钟只能是 21M ，采样周期设置为 3 个周期，算出最短的转换时间为 $0.7142\mu\text{s}$ ，这个才是最常用的。

6. ⑥数据寄存器

一切准备就绪后，ADC 转换后的数据根据转换组的不同，规则组的数据放在 ADC_DR 寄存器，注入组的数据放在 JDRx 。如果是使用双重或者三重模式那规则组的数据是存放在通用规则寄存器 ADC_CDR 内的。

规则数据寄存器 ADC_DR

ADC 规则组数据寄存器 ADC_DR 只有一个，是一个 32 位的寄存器，只有低 16 位有效并且只是用于独立模式存放转换完成数据。因为 ADC 的最大精度是 12 位， ADC_DR 是 16 位有效，这样允许 ADC 存放数据时候选择左对齐或者右对齐，具体是以哪一种方式存放，由 ADC_CR2 的 11 位 ALIGN 设置。假如设置 ADC 精度为 12 位，如果设置数据为左对齐，那 AD 转换完成数据存放在 ADC_DR 寄存器的 $[4:15]$ 位内；如果为右对齐，则存放在 ADC_DR 寄存器的 $[0:11]$ 位内。

规则通道可以有 16 个这么多，可规则数据寄存器只有一个，如果使用多通道转换，那转换的数据就全部都挤在了 DR 里面，前一个时间点转换的通道数据，就会被下一个时间点的另外一个通道转换的数据覆盖掉，所以当通道转换完成后就应该把数据取走，或者开启 DMA 模式，把数据传输到内存里面，不然就会造成数据的覆盖。最常用的做法就是开启 DMA 传输。

如果没有使用 DMA 传输，我们一般都需要使用 ADC 状态寄存器 ADC_SR 获取当前 ADC 转换的进度状态，进而进行程序控制。

注入数据寄存器 ADC_JDRx

ADC 注入组最多有 4 个通道，刚好注入数据寄存器也有 4 个，每个通道对应着自己的寄存器，不会跟规则寄存器那样产生数据覆盖的问题。 ADC_JDRx 是 32 位的，低 16 位有效，高 16 位保留，数据同样分为左对齐和右对齐，具体是以哪一种方式存放，由 ADC_CR2 的 11 位 ALIGN 设置。

通用规则数据寄存器 ADC_CDR

规则数据寄存器 ADC_DR 是仅适用于独立模式的，而通用规则数据寄存器 ADC_CDR 是适用于双重和三重模式的。独立模式就是仅仅适用三个 ADC 的其中一个，双重模式就是同时使用 ADC1 和 ADC2 ，而三重模式就是三个 ADC 同时使用。在双重或者三重模式下一般需要配合 DMA 数据传输使用。

7. ⑦中断

转换结束中断

数据转换结束后，可以产生中断，中断分为四种：规则通道转换结束中断，注入通道转换结束中断，模拟看门狗中断和溢出中断。其中转换结束中断很好理解，跟我们平时接触的中断一样，有相应的中断标志位和中断使能位，我们还可以根据中断类型写相应配套的中断服务程序。

模拟看门狗中断

当被 ADC 转换的模拟电压低于低阈值或者高于高阈值时，就会产生中断，前提是我们开启了模拟看门狗中断，其中低阈值和高阈值由 ADC_LTR 和 ADC_HTR 设置。例如我们设置高阈值是 2.5V，那么模拟电压超过 2.5V 的时候，就会产生模拟看门狗中断，反之低阈值也一样。

溢出中断

如果发生 DMA 传输数据丢失，会置位 ADC 状态寄存器 ADC_SR 的 OVR 位，如果同时使能了溢出中断，那在转换结束后会产生一个溢出中断。

DMA 请求

规则和注入通道转换结束后，除了产生中断外，还可以产生 DMA 请求，把转换好的数据直接存储在内存里面。对于独立模式的多通道 AD 转换使用 DMA 传输非常有必要，程序编程简化了很多。对于双重或三重模式使用 DMA 传输几乎可以说是必要的。有关 DMA 请求需要配合《STM32F4xx 中文参考手册》DMA 控制器这一章节来学习。一般我们在使用 ADC 的时候都会开启 DMA 传输。

8. ⑧电压转换

模拟电压经过 ADC 转换后，是一个相对精度的数字值，如果通过串口以 16 进制打印出来的话，可读性比较差，那么有时候我们就需要把数字电压转换成模拟电压，也可以跟实际的模拟电压（用万用表测）对比，看看转换是否准确。

我们一般在设计原理图的时候会把 ADC 的输入电压范围设定在：0~3.3v，如果设置 ADC 为 12 位的，那么 12 位满量程对应的就是 3.3V，12 位满量程对应的数字值是： 2^{12} 。数值 0 对应的就是 0V。如果转换后的数值为 X，X 对应的模拟电压为 Y，那么会有这么一个等式成立： $2^{12} / 3.3 = X / Y$ ， $\Rightarrow Y = (3.3 * X) / 2^{12}$ 。

31.3 ADC 初始化结构体详解

标准库函数对每个外设都建立了一个初始化结构体 `xxx_InitTypeDef` (xxx 为外设名称)，结构体成员用于设置外设工作参数，并由标准库函数 `xxx_Init()` 调用这些设定参数进入设置外设相应的寄存器，达到配置外设工作环境的目的。

结构体 `xxx_InitTypeDef` 和库函数 `xxx_Init` 配合使用是标准库精髓所在，理解了结构体 `xxx_InitTypeDef` 每个成员意义基本上就可以对该外设运用自如了。结构体 `xxx_InitTypeDef`

定义在 stm32f4xx_xxx.h 文件中，库函数 xxx_Init 定义在 stm32f4xx_xxx.c 文件中，编程时我们可以结合这两个文件内注释使用。

ADC_InitTypeDef 结构体

ADC_InitTypeDef 结构体定义在 stm32f4xx_adc.h 文件内，具体定义如下：

```
1 typedef struct {
2     uint32_t ADC_Resolution;           //ADC 分辨率选择
3     FunctionalState ADC_ScanConvMode;   //ADC 扫描选择
4     FunctionalState ADC_ContinuousConvMode; //ADC 连续转换模式选择
5     uint32_t ADC_ExternalTrigConvEdge;   //ADC 外部触发极性
6     uint32_t ADC_ExternalTrigConv;       //ADC 外部触发选择
7     uint32_t ADC_DataAlign;             //输出数据对齐方式
8     uint8_t ADC_NbrOfChannel;            //转换通道数目
9 } ADC_InitTypeDef;
```

ADC_Resolution: 配置 ADC 的分辨率，可选的分辨率有 12 位、10 位、8 位和 6 位。分辨率越高，AD 转换数据精度越高，转换时间也越长；分辨率越低，AD 转换数据精度越低，转换时间也越短。

ScanConvMode: 可选参数为 ENABLE 和 DISABLE，配置是否使用扫描。如果是单通道 AD 转换使用 DISABLE，如果是多通道 AD 转换使用 ENABLE。

ADC_ContinuousConvMode: 可选参数为 ENABLE 和 DISABLE，配置是启动自动连续转换还是单次转换。使用 ENABLE 配置为使能自动连续转换；使用 DISABLE 配置为单次转换，转换一次后停止需要手动控制才重新启动转换。

ADC_ExternalTrigConvEdge: 外部触发极性选择，如果使用外部触发，可以选择触发的极性，可选有禁止触发检测、上升沿触发检测、下降沿触发检测以及上升沿和下降沿均可触发检测。

ADC_ExternalTrigConv: 外部触发选择，图 31-1 中列举了很多外部触发条件，可根据项目需求配置触发来源。实际上，我们一般使用软件自动触发。

ADC_DataAlign: 转换结果数据对齐模式，可选右对齐 ADC_DataAlign_Right 或者左对齐 ADC_DataAlign_Left。一般我们选择右对齐模式。

ADC_NbrOfChannel: AD 转换通道数目。

ADC_CommonInitTypeDef 结构体

ADC 除了有 ADC_InitTypeDef 初始化结构体外，还有一个 ADC_CommonInitTypeDef 通用初始化结构体。ADC_CommonInitTypeDef 结构体内容决定三个 ADC 共用的工作环境，比如模式选择、ADC 时钟等等。

ADC_CommonInitTypeDef 结构体也是定义在 stm32_f4xx.h 文件中，具体定义如下：

```
1 typedef struct {
2     uint32_t ADC_Mode;           //ADC 模式选择
3     uint32_t ADC_Prescaler;       //ADC 分频系数
4     uint32_t ADC_DMAAccessMode;   //DMA 模式配置
5     uint32_t ADC_TwoSamplingDelay; //采样延迟
6 } ADC_CommonInitTypeDef;
```

ADC_Mode: ADC 工作模式选择，有独立模式、双重模式以及三重模式。

ADC_Prescaler: ADC 时钟分频系数选择, ADC 时钟是有 PCLK2 分频而来, 分频系数决定 ADC 时钟频率, 可选的分频系数为 2、4、6 和 8。ADC 最大时钟配置为 36MHz。

ADC_DMAAccessMode: DMA 模式设置, 只有在双重或者三重模式才需要设置, 可以设置三种模式, 具体可参考参考手册说明。

ADC_TwoSamplingDelay: 2 个采样阶段之前的延迟, 仅适用于双重或三重交错模式。

31.4 独立模式单通道采集实验

STM32 的 ADC 功能繁多, 我们设计三个实验尽量完整的展示 ADC 的功能。首先是比较基础实用的单通道采集, 实现开发板上电位器的动触点输出引脚电压的采集并通过串口打印至 PC 端串口调试助手。单通道采集适用 AD 转换完成中断, 在中断服务函数中读取数据, 不使用 DMA 传输, 在多通道采集时才使用 DMA 传输。

31.4.1 硬件设计

开发板板载一个贴片滑动变阻器, 电路设计见图 31-5。

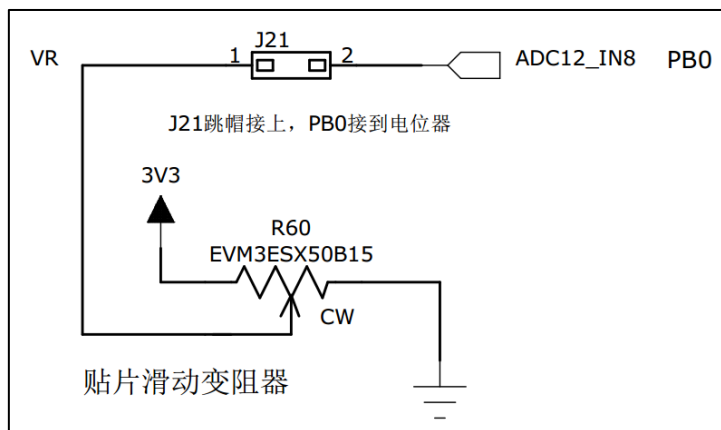


图 31-5 开发板电位器部分原理图

贴片滑动变阻器的动触点通过一个跳帽连接至 STM32 芯片的 ADC 通道引脚。当我们使用旋转滑动变阻器调节旋钮时, 其动触点电压也会随之改变, 电压变化范围为 0~3.3V, 亦是开发板默认的 ADC 电压采集范围。

31.4.2 软件设计

这里只讲解核心的部分代码, 有些变量的设置, 头文件的包含等并没有涉及到, 完整的代码请参考本章配套的工程。

我们编写两个 ADC 驱动文件, bsp_adc.h 和 bsp_adc.c, 用来存放 ADC 所用 IO 引脚的初始化函数以及 ADC 配置相关函数。

1. 编程要点

- 1) 初始化配置 ADC 目标引脚为模拟输入模式;

- 2) 使能 ADC 时钟;
- 3) 配置通用 ADC 为独立模式, 采样 4 分频;
- 4) 设置目标 ADC 为 12 位分辨率, 1 通道的连续转换, 不需要外部触发;
- 5) 设置 ADC 转换通道顺序及采样时间;
- 6) 配置使能 ADC 转换完成中断, 在中断内读取转换完数据;
- 7) 启动 ADC 转换;
- 8) 使能软件触发 ADC 转换。

ADC 转换结果数据使用中断方式读取, 这里没有使用 DMA 进行数据传输。

2. 代码分析

ADC 宏定义

代码清单 31-1 ADC 宏定义

```
1 // ADC GPIO 宏定义
2 #define RHEOSTAT_ADC_GPIO_PORT      GPIOB
3 #define RHEOSTAT_ADC_GPIO_PIN       GPIO_Pin_0
4 #define RHEOSTAT_ADC_GPIO_CLK       RCC_AHB1Periph_GPIOB
5
6 // ADC 序号宏定义
7 #define RHEOSTAT_ADC                 ADC1
8 #define RHEOSTAT_ADC_CLK             RCC_APB2Periph_ADC1
9 #define RHEOSTAT_ADC_CHANNEL         ADC_Channel_8
10
11
12 // ADC 中断宏定义, ADC123 共用一个中断源
13 #define Rheostat_ADC_IRQ             ADC_IRQn
14 #define Rheostat_ADC_INT_FUNCTION    ADC_IRQHandler
```

使用宏定义引脚信息方便硬件电路改动时程序移植。

ADC GPIO 初始化函数

代码清单 31-2 ADC GPIO 初始化

```
1 static void Rheostat_ADC_GPIO_Config(void)
2 {
3     GPIO_InitTypeDef GPIO_InitStructure;
4
5     // 使能 GPIO 时钟
6     RCC_AHB1PeriphClockCmd(RHEOSTAT_ADC_GPIO_CLK, ENABLE);
7
8     // 配置 IO
9     GPIO_InitStructure.GPIO_Pin = RHEOSTAT_ADC_GPIO_PIN;
10    // 配置为模拟输入
11    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AIN;
12    // 不上拉不下拉
13    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL ;
14    GPIO_Init(RHEOSTAT_ADC_GPIO_PORT, &GPIO_InitStructure);
```

使用到 GPIO 时候都必须开启对应的 GPIO 时钟，GPIO 用于 AD 转换功能必须配置为模拟输入模式。

配置 ADC 工作模式

代码清单 31-3 ADC 工作模式配置

```
1 static void Rheostat_ADC_Mode_Config(void)
2 {
3     ADC_InitTypeDef ADC_InitStructure;
4     ADC_CommonInitTypeDef ADC_CommonInitStructure;
5
6     // 开启 ADC 时钟
7     RCC_APB2PeriphClockCmd(RHEOSTAT_ADC_CLK , ENABLE);
8
9     // -----ADC Common 结构体 参数 初始化-----
10    // 独立 ADC 模式
11    ADC_CommonInitStructure.ADC_Mode = ADC_Mode_Independent;
12    // 时钟为 fpclock x 分频
13    ADC_CommonInitStructure.ADC_Prescaler = ADC_Prescaler_Div2;
14    // 禁止 DMA 直接访问模式
15    ADC_CommonInitStructure.ADC_DMAAccessMode=ADC_DMAAccessMode_Disabled;
16    // 采样时间间隔
17    ADC_CommonInitStructure.ADC_TwoSamplingDelay=
18        ADC_TwoSamplingDelay_20Cycles;
19    ADC_CommonInit(&ADC_CommonInitStructure);
20
21    // -----ADC Init 结构体 参数 初始化-----
22    // ADC 分辨率
23    ADC_InitStructure.ADC_Resolution = ADC_Resolution_12b;
24    // 禁止扫描模式，多通道采集才需要
25    ADC_InitStructure.ADC_ScanConvMode = DISABLE;
26    // 连续转换
27    ADC_InitStructure.ADC_ContinuousConvMode = ENABLE;
28    //禁止外部边沿触发
29    ADC_InitStructure.ADC_ExternalTrigConvEdge =
30        ADC_ExternalTrigConvEdge_None;
31    //使用软件触发，外部触发不用配置，注释掉即可
32    //ADC_InitStructure.ADC_ExternalTrigConv=ADC_ExternalTrigConv_T1_CC1;
33    //数据右对齐
34    ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right;
35    //转换通道 1 个
36    ADC_InitStructure.ADC_NbrOfConversion = 1;
37    ADC_Init(RHEOSTAT_ADC, &ADC_InitStructure);
38    //-----
39    // 配置 ADC 通道转换顺序和采样周期
40    ADC_RegularChannelConfig(RHEOSTAT_ADC, RHEOSTAT_ADC_CHANNEL,
41        1, ADC_SampleTime_56Cycles);
42
43    // ADC 转换结束产生中断，在中断服务程序中读取转换值
44    ADC_ITConfig(RHEOSTAT_ADC, ADC_IT_EOC, ENABLE);
45    // 使能 ADC
46    ADC_Cmd(RHEOSTAT_ADC, ENABLE);
47    //开始 adc 转换，软件触发
48    ADC_SoftwareStartConv(RHEOSTAT_ADC);
49 }
```

首先，使用 ADC_InitTypeDef 和 ADC_CommonInitTypeDef 结构体分别定义一个 ADC 初始化和 ADC 通用类型变量，这两个结构体我们之前已经有详细讲解。

我们调用 RCC_APB2PeriphClockCmd()开启 ADC 时钟。

接下来我们使用 `ADC_CommonInitTypeDef` 结构体变量 `ADC_CommonInitStructure` 来配置 ADC 为独立模式、分频系数为 2、不需要设置 DMA 模式、20 个周期的采样延迟，并调用 `ADC_CommonInit` 函数完成 ADC 通用工作环境配置。

我们使用 `ADC_InitTypeDef` 结构体变量 `ADC_InitStructure` 来配置 ADC1 为 12 位分辨率、单通道采集不需要扫描、启动连续转换、使用内部软件触发无需外部触发事件、使用右对齐数据格式、转换通道为 1，并调用 `ADC_Init` 函数完成 ADC1 工作环境配置。

`ADC_RegularChannelConfig` 函数用来绑定 ADC 通道转换顺序和时间。它接收 4 个形参，第一个形参选择 ADC 外设，可为 ADC1、ADC2 或 ADC3；第二个形参通道选择，总共可选 18 个通道；第三个形参为转换顺序，可选为 1 到 16；第四个形参为采样周期选择，采样周期越短，ADC 转换数据输出周期就越短但数据精度也越低，采样周期越长，ADC 转换数据输出周期就越长同时数据精度越高。PC3 对应 ADC 通道 `ADC_Channel_13`，这里我们选择 `ADC_SampleTime_56Cycles` 即 56 周期的采样时间。

利用 ADC 转换完成中断可以非常方便的保证我们读取到的数据是转换完成后的数据而不用担心该数据可能是 ADC 正在转换时“不稳定”的数据。我们使用 `ADC_ITConfig` 函数使能 ADC 转换完成中断，并在中断服务函数中读取转换结果数据。

`ADC_Cmd` 函数控制 ADC 转换启动和停止。

最后，如果使用软件触发需要调用 `ADC_SoftwareStartConvCmd` 函数进行使能配置。

ADC 中断配置

代码清单 31-4 ADC 中断配置

```
1 static void Rheostat_ADC_NVIC_Config(void)
2 {
3     NVIC_InitTypeDef NVIC_InitStructure;
4     /* 配置优先级分组 */
5     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_1);
6     /* 配置中断源 */
7     NVIC_InitStructure.NVIC_IRQChannel = Rheostat_ADC_IRQ;
8     NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;
9     NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
10    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
11
12    NVIC_Init(&NVIC_InitStructure);
13 }
```

在 `Rheostat_ADC_NVIC_Config` 函数中我们配置了 ADC 转换完成中断的优先级分组和优先级配置。

ADC 中断服务函数

代码清单 31-5 ADC 中断服务函数

```
1 void ADC_IRQHandler(void)
2 {
3     if (ADC_GetITStatus(RHEOSTAT_ADC, ADC_IT_EOC) == SET) {
4         // 读取 ADC 的转换值
5         ADC_ConvertedValue = ADC_GetConversionValue(RHEOSTAT_ADC);
6     }
7 }
8 ADC_ClearITPendingBit(RHEOSTAT_ADC, ADC_IT_EOC);
```

```
9
10 }
```

中断服务函数一般定义在 `stm32f4xx_it.c` 文件内，我们只使能了 ADC 转换完成中断，在 ADC 转换完成后就会进入中断服务函数，我们在中断服务函数内直接读取 ADC 转换结果保存在变量 `ADC_ConvertedValue` (在 `main.c` 中定义) 中。

`ADC_GetConversionValue` 函数是获取 ADC 转换结果值的库函数，只有一个形参为 ADC 外设，可选为 `ADC1`、`ADC2` 或 `ADC3`，该函数还返回一个 16 位的 ADC 转换结果值。

主函数

代码清单 31-6 主函数

```
1 int main(void)
2 {
3     /*初始化 USART1*/
4     Debug_USART_Config();
5
6     /* 初始化滑动变阻器用到的 DAC，ADC 采集完成会产生 ADC 中断，
7     在 stm32f4xx_it.c 文件中的中断服务函数更新 ADC_ConvertedValue 的值 */
8     Rheostat_Init();
9
10    printf("\r\n ----这是一个 ADC 实验(NO DMA 传输)----\r\n");
11
12
13    while (1) {
14        Delay(0xffffee);
15        printf("\r\n The current AD value = 0x%04X \r\n",
16               ADC_ConvertedValue);
17        printf("\r\n The current AD value = %f V \r\n", ADC_Vol);
18
19        ADC_Vol = (float) (ADC_ConvertedValue*3.3/4096); // 读取转换的 AD 值
20
21    }
22 }
```

主函数先调用 `USARTx_Config` 函数配置调试串口相关参数，函数定义在 `bsp_debug_usart.c` 文件中。

接下来调用 `Rheostat_Init` 函数进行 ADC 初始化配置并启动 ADC。`Rheostat_Init` 函数是定义在 `bsp_adc.c` 文件中，它只是简单的分别调用 `Rheostat_ADC_GPIO_Config()`、`Rheostat_ADC_Mode_Config()` 和 `Rheostat_ADC_NVIC_Config()`。

`Delay` 函数只是一个简单的延时函数。

在 ADC 中断服务函数中我们把 AD 转换结果保存在变量 `ADC_ConvertedValue` 中，根据我们之前的分析可以非常清楚的计算出对应的电位器动触点的电压值。

最后就是把相关数据打印至串口调试助手。

31.4.3 下载验证

用 USB 线连接开发板“USB 转串口”接口跟电脑，在电脑端打开串口调试助手，把编译好的程序下载到开发板。在串口调试助手可看到不断有数据从开发板传输过来，此时我们旋转电位器改变其电阻值，那么对应的数据也会有变化。

31.5 独立模式多通道采集实验

31.5.1 硬件设计

开发板已通过排针接口把部分 ADC 通道引脚引出，我们可以根据需要选择使用。实际使用时候必须注意保存 ADC 引脚是单独使用的，不可能与其他模块电路共用同一引脚。

31.5.2 软件设计

这里只讲解核心的部分代码，有些变量的设置，头文件的包含等并没有涉及到，完整的代码请参考本章配套的工程。

跟单通道例程一样，我们编写两个 ADC 驱动文件，bsp_adc.h 和 bsp_adc.c，用来存放 ADC 所用 IO 引脚的初始化函数以及 ADC 配置相关函数，实际上这两个文件跟单通道实验的文件是非常相似的。

1. 编程要点

- 1) 初始化配置 ADC 目标引脚为模拟输入模式；
- 2) 使能 ADC 时钟和 DMA 时钟；
- 3) 配置 DMA 从 ADC 数据寄存器传输数据到我们指定的存储区；
- 4) 配置通用 ADC 为独立模式，采样 4 分频；
- 5) 设置 ADC 为 12 位分辨率，启动扫描，连续转换，不需要外部触发；
- 6) 设置 ADC 转换通道顺序及采样时间；
- 7) 使能 DMA 请求，DMA 在 AD 转换完自动传输数据到指定的存储区；
- 8) 启动 ADC 转换；
- 9) 使能软件触发 ADC 转换。

ADC 转换结果数据使用 DMA 方式传输至指定的存储区，这样取代单通道实验使用中断服务的读取方法。实际上，多通道 ADC 采集一般使用 DMA 数据传输方式更加高效方便。

2. 代码分析

ADC 宏定义

代码清单 31-7 多通道 ADC 相关宏定义

```
1 #define RHEOSTAT_NOFCHANNEL      3
2
3 /*=====通道 1 IO=====*/
```



```
4 // PB0 通过调幅接电位器
5 // ADC IO 宏定义
6 #define RHEOSTAT_ADC_GPIO_PORT1      GPIOB
7 #define RHEOSTAT_ADC_GPIO_PIN1      GPIO_Pin_0
8 #define RHEOSTAT_ADC_GPIO_CLK1      RCC_AHB1Periph_GPIOB
9 #define RHEOSTAT_ADC_CHANNEL1      ADC_Channel_8
10 /*=====通道 2 IO =====*/
11 // PB1 通过调幅接光敏电阻
12 // ADC IO 宏定义
13 #define RHEOSTAT_ADC_GPIO_PORT2      GPIOB
14 #define RHEOSTAT_ADC_GPIO_PIN2      GPIO_Pin_1
15 #define RHEOSTAT_ADC_GPIO_CLK2      RCC_AHB1Periph_GPIOB
16 #define RHEOSTAT_ADC_CHANNEL2      ADC_Channel_9
17 /*=====通道 3 IO =====*/
18 // PA6 悬空, 可用杜邦线接 3V3 或者 GND 来实验
19 // ADC IO 宏定义
20 #define RHEOSTAT_ADC_GPIO_PORT3      GPIOA
21 #define RHEOSTAT_ADC_GPIO_PIN3      GPIO_Pin_6
22 #define RHEOSTAT_ADC_GPIO_CLK3      RCC_AHB1Periph_GPIOA
23 #define RHEOSTAT_ADC_CHANNEL3      ADC_Channel_6
24
25 // ADC 序号宏定义
26 #define RHEOSTAT_ADC                  ADC1
27 #define RHEOSTAT_ADC_CLK              RCC_APB2Periph_ADC1
28 // ADC DR 寄存器宏定义, ADC 转换后的数字值则存放在这里
29 #define RHEOSTAT_ADC_DR_ADDR          ((u32)ADC1+0x4c)
30
31
32 // ADC DMA 通道宏定义, 这里我们使用 DMA 传输
33 #define RHEOSTAT_ADC_DMA_CLK          RCC_AHB1Periph_DMA2
34 #define RHEOSTAT_ADC_DMA_CHANNEL      DMA_Channel_0
35 #define RHEOSTAT_ADC_DMA_STREAM      DMA2_Stream0
```

定义多个通道进行多通道 ADC 实验, 并且定义 DMA 相关配置。

ADC GPIO 初始化函数

代码清单 31-8 ADC GPIO 初始化

```
1 static void Rheostat_ADC_GPIO_Config(void)
2 {
3     GPIO_InitTypeDef GPIO_InitStructure;
4     /*=====通道 1=====*/
5     // 使能 GPIO 时钟
6     RCC_AHB1PeriphClockCmd(RHEOSTAT_ADC_GPIO_CLK1,ENABLE);
7     // 配置 IO
8     GPIO_InitStructure.GPIO_Pin = RHEOSTAT_ADC_GPIO_PIN1;
9     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AIN;
10    //不上拉不下拉
11    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL ;
12    GPIO_Init(RHEOSTAT_ADC_GPIO_PORT1, &GPIO_InitStructure);
13
14    /*=====通道 2=====*/
15    // 使能 GPIO 时钟
16    RCC_AHB1PeriphClockCmd(RHEOSTAT_ADC_GPIO_CLK2,ENABLE);
17    // 配置 IO
18    GPIO_InitStructure.GPIO_Pin = RHEOSTAT_ADC_GPIO_PIN2;
19    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AIN;
20    //不上拉不下拉
21    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL ;
22    GPIO_Init(RHEOSTAT_ADC_GPIO_PORT2, &GPIO_InitStructure);
23
24    /*=====通道 3=====*/
25    // 使能 GPIO 时钟
```

```
26 RCC_AHB1PeriphClockCmd(RHEOSTAT_ADC_GPIO_CLK3, ENABLE);
27 // 配置 IO
28 GPIO_InitStructure.GPIO_Pin = RHEOSTAT_ADC_GPIO_PIN3;
29 GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AIN;
30 //不上拉不下拉
31 GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL ;
32 GPIO_Init(RHEOSTAT_ADC_GPIO_PORT3, &GPIO_InitStructure);
33 }
```

使用到 GPIO 时候都必须开启对应的 GPIO 时钟，GPIO 用于 AD 转换功能必须配置为模拟输入模式。

配置 ADC 工作模式

代码清单 31-9 ADC 工作模式配置

```
1 static void Rheostat_ADC_Mode_Config(void)
2 {
3     DMA_InitTypeDef DMA_InitStructure;
4     ADC_InitTypeDef ADC_InitStructure;
5     ADC_CommonInitTypeDef ADC_CommonInitStructure;
6
7     // -----DMA Init 结构体参数 初始化-----
8     // ADC1 使用 DMA2，数据流 0，通道 0，这个是手册固定死的
9     // 开启 DMA 时钟
10    RCC_AHB1PeriphClockCmd(RHEOSTAT_ADC_DMA_CLK, ENABLE);
11    // 外设基址为：ADC 数据寄存器地址
12    DMA_InitStructure.DMA_PeripheralBaseAddr = RHEOSTAT_ADC_DR_ADDR;
13    // 存储器地址，实际上就是一个内部 SRAM 的变量
14    DMA_InitStructure.DMA_Memory0BaseAddr = (u32)ADC_ConvertedValue;
15    // 数据传输方向为外设到存储器
16    DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralToMemory;
17    // 缓冲区大小为，指一次传输的数据量
18    DMA_InitStructure.DMA_BufferSize = RHEOSTAT_NOFCHANNEL;
19    // 外设寄存器只有一个，地址不用递增
20    DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
21    // 存储器地址固定
22    DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;
23    // // 外设数据大小为半字，即两个字节
24    DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_HalfWord;
25    // 存储器数据大小也为半字，跟外设数据大小相同
26    DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_HalfWord;
27    // 循环传输模式
28    DMA_InitStructure.DMA_Mode = DMA_Mode_Circular;
29    // DMA 传输通道优先级为高，当使用一个 DMA 通道时，优先级设置不影响
30    DMA_InitStructure.DMA_Priority = DMA_Priority_High;
31    // 禁止 DMA FIFO，使用直连模式
32    DMA_InitStructure.DMA_FIFOMode = DMA_FIFOMode_Disable;
33    // FIFO 大小，FIFO 模式禁止时，这个不用配置
34    DMA_InitStructure.DMA_FIFOThreshold = DMA_FIFOThreshold_HalfFull;
35    DMA_InitStructure.DMA_MemoryBurst = DMA_MemoryBurst_Single;
36    DMA_InitStructure.DMA_PeripheralBurst = DMA_PeripheralBurst_Single;
37    // 选择 DMA 通道，通道存在于流中
38    DMA_InitStructure.DMA_Channel = RHEOSTAT_ADC_DMA_CHANNEL;
39    //初始化 DMA 流，流相当于一个大的管道，管道里面有很多通道
40    DMA_Init(RHEOSTAT_ADC_DMA_STREAM, &DMA_InitStructure);
41    // 使能 DMA 流
42    DMA_Cmd(RHEOSTAT_ADC_DMA_STREAM, ENABLE);
43
44    // 开启 ADC 时钟
45    RCC_APB2PeriphClockCmd(RHEOSTAT_ADC_CLK, ENABLE);
46    // -----ADC Common 结构体 参数 初始化-----
47    // 独立 ADC 模式
```

```
48     ADC_CommonInitStructure.ADC_Mode = ADC_Mode_Independent;
49     // 时钟为 fpcclk x 分频
50     ADC_CommonInitStructure.ADC_Prescaler = ADC_Prescaler_Div4;
51     // 禁止 DMA 直接访问模式
52     ADC_CommonInitStructure.ADC_DMAAccessMode = ADC_DMAAccessMode_Disabled;
53     // 采样时间间隔
54     ADC_CommonInitStructure.ADC_TwoSamplingDelay = ADC_TwoSamplingDelay_20Cycles;
55     ADC_CommonInit(&ADC_CommonInitStructure);
56
57     // -----ADC Init 结构体 参数 初始化-----
58     ADC_StructInit(&ADC_InitStructure);
59     // ADC 分辨率
60     ADC_InitStructure.ADC_Resolution = ADC_Resolution_12b;
61     // 扫描模式, 多通道采集需要
62     ADC_InitStructure.ADC_ScanConvMode = ENABLE;
63     // 连续转换
64     ADC_InitStructure.ADC_ContinuousConvMode = ENABLE;
65     //禁止外部边沿触发
66     ADC_InitStructure.ADC_ExternalTrigConvEdge = ADC_ExternalTrigConvEdge_None;
67     //外部触发通道, 本例子使用软件触发, 此值随便赋值即可
68     ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_T1_CC1;
69     //数据右对齐
70     ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right;
71     //转换通道 1 个
72     ADC_InitStructure.ADC_NbrOfConversion = RHEOSTAT_NOFCHANNEL;
73     ADC_Init(RHEOSTAT_ADC, &ADC_InitStructure);
74     //-----
75
76     // 配置 ADC 通道转换顺序和采样时间周期
77     ADC-RegularChannelConfig(RHEOSTAT_ADC, RHEOSTAT_ADC_CHANNEL1, 1,
78                             ADC_SampleTime_3Cycles);
79     ADC-RegularChannelConfig(RHEOSTAT_ADC, RHEOSTAT_ADC_CHANNEL2, 2,
80                             ADC_SampleTime_3Cycles);
81     ADC-RegularChannelConfig(RHEOSTAT_ADC, RHEOSTAT_ADC_CHANNEL3, 3,
82                             ADC_SampleTime_3Cycles);
83
84     // 使能 DMA 请求 after last transfer (Single-ADC mode)
85     ADC_DMAResquestAfterLastTransferCmd(RHEOSTAT_ADC, ENABLE);
86     // 使能 ADC DMA
87     ADC_DMACmd(RHEOSTAT_ADC, ENABLE);
88
89     // 使能 ADC
90     ADC_Cmd(RHEOSTAT_ADC, ENABLE);
91     //开始 adc 转换, 软件触发
92     ADC_SoftwareStartConv(RHEOSTAT_ADC);
93 }
```

首先, 我们使用了 `DMA_InitTypeDef` 定义了一个 DMA 初始化类型变量, 该结构体内容我们在 DMA 篇已经做了非常详细的讲解; 另外还使用 `ADC_InitTypeDef` 和 `ADC_CommonInitTypeDef` 结构体分别定义一个 ADC 初始化和 ADC 通用类型变量, 这两个结构体我们之前已经有详细讲解。

调用 `RCC_APB2PeriphClockCmd()` 开启 ADC 时钟以及 `RCC_AHB1PeriphClockCmd()` 开启 DMA 时钟。

我们需要对 DMA 进行必要的配置。首先设置外设基地址就是 ADC 的规则数据寄存器地址; 存储器的地址就是我们指定的数据存储区空间, `ADC_ConvertedValue` 是我们定义的一个全局数组名, 它是一个无符号 16 位含有 4 个元素的整数数组; ADC 规则转换对应只有一个数据寄存器所以地址不能递增, 而我们定义的存储区是专门用来存放不同通道数据的, 所以需要自动地址递增。ADC 的规则数据寄存器只有低 16 位有效, 实际存放的数据

只有 12 位而已，所以设置数据大小为半字大小。ADC 配置为连续转换模式 DMA 也设置为循环传输模式。设置好 DMA 相关参数后就使能 DMA 的 ADC 通道。

接下来我们使用 ADC_CommonInitTypeDef 结构体变量 ADC_CommonInitStructure 来配置 ADC 为独立模式、分频系数为 4、不需要设置 DMA 模式、20 个周期的采样延迟，并调用 ADC_CommonInit 函数完成 ADC 通用工作环境配置。

我们使用 ADC_InitTypeDef 结构体变量 ADC_InitStructure 来配置 ADC1 为 12 位分辨率、使能扫描模式、启动连续转换、使用内部软件触发无需外部触发事件、使用右对齐数据格式、转换通道为 4，并调用 ADC_Init 函数完成 ADC3 工作环境配置。

ADC_RegularChannelConfig 函数用来绑定 ADC 通道转换顺序和采样时间。分别绑定四个 ADC 通道引脚并设置相应的转换顺序。

ADC_DMARequestAfterLastTransferCmd 函数控制是否使能 ADC 的 DMA 请求，如果使能请求，并调用 ADC_DMACmd 函数使能 DMA，则在 ADC 转换完成后就请求 DMA 实现数据传输。

ADC_Cmd 函数控制 ADC 转换启动和停止。

最后，如果使用软件触发需要调用 ADC_SoftwareStartConvCmd 函数进行使能配置。

主函数

代码清单 31-10 主函数

```
1 #include "stm32f4xx.h"
2 #include "../usart/bsp_debug_usart.h"
3 #include "../adc/bsp_adc.h"
4
5 // ADC 转换的电压值通过 MDA 方式传到 SRAM
6 extern __IO uint16_t ADC_ConvertedValue[RHEOSTAT_NOFCHANNEL];
7
8 // 局部变量，用于保存转换计算后的电压值
9 float ADC_ConvertedValueLocal[RHEOSTAT_NOFCHANNEL] = {0};
10
11 static void Delay(__IO uint32_t nCount) //简单的延时函数
12 {
13     for (; nCount != 0; nCount--);
14 }
15
16 /**
17  * @brief 主函数
18  * @param 无
19  * @retval 无
20  */
21 int main(void)
22 {
23     /*初始化 USART 配置模式为 115200 8-N-1，中断接收*/
24     Debug_USART_Config();
25     Rheostat_Init();
26
27     while (1) {
28         ADC_ConvertedValueLocal[0] = (float) ADC_ConvertedValue[0]/4096*(float)3.3;
29         ADC_ConvertedValueLocal[1] = (float) ADC_ConvertedValue[1]/4096*(float)3.3;
30         ADC_ConvertedValueLocal[2] = (float) ADC_ConvertedValue[2]/4096*(float)3.3;
31
32         printf("\r\n CH1_C3 value = %f V \r\n",ADC_ConvertedValueLocal[0]);
33         printf("\r\n CH2_PA4 value = %f V \r\n",ADC_ConvertedValueLocal[1]);
34         printf("\r\n CH3_PA6 value = %f V \r\n",ADC_ConvertedValueLocal[2]);
35     }
```

```
36     printf("\r\n\r\n");
37     Delay(0xfffffff);
38 }
39 }
```

主函数先调用 USARTx_Config 函数配置调试串口相关参数，函数定义在 bsp_debug_usart.c 文件中。

接下来调用 Rheostat_Init 函数进行 ADC 初始化配置并启动 ADC。Rheostat_Init 函数是定义在 bsp_adc.c 文件中，它只是简单的分别调用 Rheostat_ADC_GPIO_Config() 和 Rheostat_ADC_Mode_Config()。

Delay 函数只是一个简单的延时函数。

我们配置了 DMA 数据传输所以它会自动把 ADC 转换完成后数据保存到数组 ADC_ConvertedValue 内，我们只要直接使用数组就可以了。经过简单地计算就可以得到每个通道对应的实际电压。

最后就是把相关数据打印至串口调试助手。

31.5.3 下载验证

将待测电压通过杜邦线接在对应引脚上，用 USB 线连接开发板“USB 转串口”接口跟电脑，在电脑端打开串口调试助手，把编译好的程序下载到开发板。在串口调试助手可看到不断有数据从开发板传输过来，此时我们改变输入电压值，那么对应的数据也会有变化。

31.6 三重 ADC 交替模式采集实验

AD 转换包括采样阶段和转换阶段，在采样阶段才对通道数据进行采集；而在转换阶段只是将采集到的数据进行转换为数字量输出，此刻通道数据变化不会改变转换结果。独立模式的 ADC 采集需要在一个通道采集并且转换完成后才会进行下一个通道的采集。双重或者三重 ADC 的机制使用两个或以上 ADC 同时采样两个或以上不同通道的数据或者使用两个或以上 ADC 交叉采集同一通道的数据。双重或者三重 ADC 模式较独立模式一个最大的优势就是转换速度快。

我们这里只介绍三重 ADC 交替模式，关于双重或者三重 ADC 的其他模式与之类似，可以参考三重 ADC 交替模式使用。三重 ADC 交替模式是针对同一通道的使用三个 ADC 交叉采集，就是在 ADC1 采样完等几个时钟周期后 ADC2 开始采样，此时 ADC1 处在转换阶段，当 ADC2 采样完成再等几个时钟周期后 ADC3 就进行采样此时 ADC1 和 ADC2 处在转换阶段，如果 ADC3 采样完成并且 ADC1 已经转换完成那么就可以准备下一轮的循环，这样充分利用转换阶段时间达到增快采样速度的效果。AD 转换过程见图 31-6，利用 ADC 的转换阶段时间另外一个 ADC 进行采样，而不用像独立模式必须等待采样和转换结束后才进行下一次采样及转换。

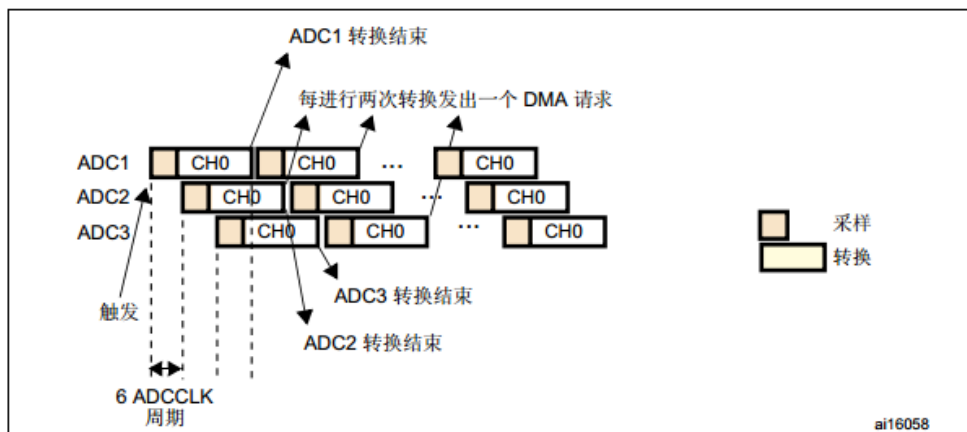


图 31-6 三重 ADC 交叉模式

31.6.1 硬件设计

三重 ADC 交叉模式是针对同一个通道的 ADC 采集模式，这种情况跟 31.4 31.4 小节的单通道实验非常类似，只是同时使用三个 ADC 对同一通道进行采集，所以电路设计与之相同即可，具体可参考图 31-5。

31.6.2 软件设计

这里只讲解核心的部分代码，有些变量的设置，头文件的包含等并没有涉及到，完整的代码请参考本章配套的工程。

跟单通道例程一样，我们编写两个 ADC 驱动文件，bsp_adc.h 和 bsp_adc.c，用来存放 ADC 所用 IO 引脚的初始化函数以及 ADC 配置相关函数，实际上这两个文件跟单通道实验的文件非常相似。

1. 编程要点

- 1) 初始化配置 ADC 目标引脚为模拟输入模式；
- 2) 使能 ADC1、ADC2、ADC3 以及 DMA 时钟；
- 3) 配置 DMA 控制将 ADC 通用数据寄存器数据转存到指定存储区；
- 4) 配置通用 ADC 为三重 ADC 交替模式，采样 4 分频，使用 DMA 模式 2；
- 5) 设置 ADC1、ADC2 和 ADC3 为 12 位分辨率，禁用扫描，连续转换，不需要外部触发；
- 6) 设置 ADC1、ADC2 和 ADC3 转换通道顺序及采样时间；
- 7) 使能 ADC1 的 DMA 请求，在 ADC 转换完后自动请求 DMA 进行数据传输；

- 8) 启动 ADC1、ADC2 和 ADC3 转换；
- 9) 使能软件触发 ADC 转换。

ADC 转换结果数据使用 DMA 方式传输至指定的存储区，这样取代单通道实验使用中断服务的读取方法。

2. 代码分析

ADC 宏定义

代码清单 31-11 三重 ADC 相关宏定义

```
1 // ADC GPIO 宏定义
2 #define RHEOSTAT_ADC_GPIO_PORT      GPIOC
3 #define RHEOSTAT_ADC_GPIO_PIN       GPIO_Pin_2
4 #define RHEOSTAT_ADC_GPIO_CLK       RCC_AHB1Periph_GPIOC
5
6 // ADC 序号宏定义
7 #define RHEOSTAT_ADC1                ADC1
8 #define RHEOSTAT_ADC1_CLK            RCC_APB2Periph_ADC1
9 #define RHEOSTAT_ADC2                ADC2
10 #define RHEOSTAT_ADC2_CLK            RCC_APB2Periph_ADC2
11 #define RHEOSTAT_ADC3                ADC3
12 #define RHEOSTAT_ADC3_CLK            RCC_APB2Periph_ADC3
13 #define RHEOSTAT_ADC_CHANNEL         ADC_Channel_12
14 // ADC CDR 寄存器宏定义，ADC 转换后的数字值则存放在这里
15 #define RHEOSTAT_ADC_CDR_ADDR        ((uint32_t)0x40012308)
16
17 // ADC DMA 通道宏定义，这里我们使用 DMA 传输
18 #define RHEOSTAT_ADC_DMA_CLK          RCC_AHB1Periph_DMA2
19 #define RHEOSTAT_ADC_DMA_CHANNEL      DMA_Channel_0
20 #define RHEOSTAT_ADC_DMA_STREAM       DMA2_Stream0
```

双重或者三重 ADC 需要使用通用规则数据寄存器 ADC_CDR，这点跟独立模式不同。定义电位器动触点引脚作为三重 ADC 的模拟输入，三个 ADC 都是采集同一路信号。

ADC GPIO 初始化函数

代码清单 31-12 ADC GPIO 初始化

```
1 static void Rheostat_ADC_GPIO_Config(void)
2 {
3     GPIO_InitTypeDef GPIO_InitStructure;
4
5     // 使能 GPIO 时钟
6     RCC_AHB1PeriphClockCmd(RHEOSTAT_ADC_GPIO_CLK, ENABLE);
7
8     // 配置 IO
9     GPIO_InitStructure.GPIO_Pin = RHEOSTAT_ADC_GPIO_PIN;
10    // 配置为模拟输入
11    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AIN;
12    // 不上拉不下拉
13    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL ;
14    GPIO_Init(RHEOSTAT_ADC_GPIO_PORT, &GPIO_InitStructure);
15 }
```

使用到 GPIO 时候都必须开启对应的 GPIO 时钟，GPIO 用于 AD 转换功能必须配置为模拟输入模式。

配置三重 ADC 交替模式

代码清单 31-13 三重 ADC 交替模式配置

```
1 static void Rheostat_ADC_Mode_Config(void)
2 {
3     DMA_InitTypeDef DMA_InitStructure;
4     ADC_InitTypeDef ADC_InitStructure;
5     ADC_CommonInitTypeDef ADC_CommonInitStructure;
6
7     // -----DMA Init 结构体参数 初始化-----
8     // ADC1 使用 DMA2，数据流 0，通道 0，这个是手册固定死的
9     // 开启 DMA 时钟
10    RCC_AHB1PeriphClockCmd(RHEOSTAT_ADC_DMA_CLK, ENABLE);
11    // 外设基址为：ADC 数据寄存器地址
12    DMA_InitStructure.DMA_PeripheralBaseAddr = RHEOSTAT_ADC_CDR_ADDR;
13    // 存储器地址，实际上就是一个内部 SRAM 的变量
14    DMA_InitStructure.DMA_Memory0BaseAddr = (u32)ADC_ConvertedValue;
15    // 数据传输方向为外设到存储器
16    DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralToMemory;
17    // 缓冲区大小为，指一次传输的数据量
18    DMA_InitStructure.DMA_BufferSize = 3;
19    // 外设寄存器只有一个，地址不用递增
20    DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
21    // 存储器地址固定
22    DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;
23    // // 外设数据大小为半字，即两个字节
24    DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Word;
25    // 存储器数据大小也为半字，跟外设数据大小相同
26    DMA_InitStructure.DMA_MemoryDataSize = DMA_PeripheralDataSize_Word;
27    // 循环传输模式
28    DMA_InitStructure.DMA_Mode = DMA_Mode_Circular;
29    // DMA 传输通道优先级为高，当使用一个 DMA 通道时，优先级设置不影响
30    DMA_InitStructure.DMA_Priority = DMA_Priority_High;
31    // 禁止 DMA FIFO，使用直连模式
32    DMA_InitStructure.DMA_FIFOMode = DMA_FIFOMode_Disable;
33    // FIFO 大小，FIFO 模式禁止时，这个不用配置
34    DMA_InitStructure.DMA_FIFOThreshold = DMA_FIFOThreshold_HalfFull;
35    DMA_InitStructure.DMA_MemoryBurst = DMA_MemoryBurst_Single;
36    DMA_InitStructure.DMA_PeripheralBurst = DMA_PeripheralBurst_Single;
37    // 选择 DMA 通道，通道存在于流中
38    DMA_InitStructure.DMA_Channel = RHEOSTAT_ADC_DMA_CHANNEL;
39    //初始化 DMA 流，流相当于一个大的管道，管道里面有很多通道
40    DMA_Init(RHEOSTAT_ADC_DMA_STREAM, &DMA_InitStructure);
41    // 使能 DMA 流
42    DMA_Cmd(RHEOSTAT_ADC_DMA_STREAM, ENABLE);
43
44    // 开启 ADC 时钟
45    RCC_APB2PeriphClockCmd(RHEOSTAT_ADC1_CLK, ENABLE);
46    RCC_APB2PeriphClockCmd(RHEOSTAT_ADC2_CLK, ENABLE);
47    RCC_APB2PeriphClockCmd(RHEOSTAT_ADC3_CLK, ENABLE);
48
49    // -----ADC Init 结构体 参数 初始化-----
50    ADC_StructInit(&ADC_InitStructure);
51    // 独立 ADC 模式
52    ADC_CommonInitStructure.ADC_Mode = ADC_TripleMode_Interl;
53    // 时钟为 fpclock x 分频
54    ADC_CommonInitStructure.ADC_Prescaler = ADC_Prescaler_Div4;
55    // 禁止 DMA 直接访问模式
```

```
56 ADC_CommonInitStructure.ADC_DMAAccessMode = ADC_DMAAccessMode_2;
57 // 采样时间间隔
58 ADC_CommonInitStructure.ADC_TwoSamplingDelay = ADC_TwoSamplingDelay_20Cycles;
59 ADC_CommonInit(&ADC_CommonInitStructure);
60
61 // -----ADC Init 结构体 参数 初始化-----
62 // ADC 分辨率
63 ADC_InitStructure.ADC_Resolution = ADC_Resolution_12b;
64 // 禁止扫描模式, 多通道采集才需要
65 ADC_InitStructure.ADC_ScanConvMode = DISABLE;
66 // 连续转换
67 ADC_InitStructure.ADC_ContinuousConvMode = ENABLE;
68 //禁止外部边沿触发
69 ADC_InitStructure.ADC_ExternalTrigConvEdge = ADC_ExternalTrigConvEdge_None;
70 //外部触发通道, 本例子使用软件触发, 此值随便赋值即可
71 ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_T1_CC1;
72 //数据右对齐
73 ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right;
74 //转换通道 1 个
75 ADC_InitStructure.ADC_NbrOfConversion = 1;
76
77 ADC_Init(RHEOSTAT_ADC1, &ADC_InitStructure);
78 //-----
79
80 // 配置 ADC 通道转换顺序为 1, 第一个转换, 采样时间为 3 个时钟周期
81 ADC_RegularChannelConfig(RHEOSTAT_ADC1, RHEOSTAT_ADC_CHANNEL, 1, ADC_SampleTime_3Cycles);
82 //-----
83
84 ADC_Init(RHEOSTAT_ADC2, &ADC_InitStructure);
85 // 配置 ADC 通道转换顺序为 1, 第一个转换, 采样时间为 3 个时钟周期
86 ADC_RegularChannelConfig(RHEOSTAT_ADC2, RHEOSTAT_ADC_CHANNEL, 1, ADC_SampleTime_3Cycles);
87 //-----
88
89 ADC_Init(RHEOSTAT_ADC3, &ADC_InitStructure);
90 // 配置 ADC 通道转换顺序为 1, 第一个转换, 采样时间为 3 个时钟周期
91 ADC_RegularChannelConfig(RHEOSTAT_ADC3, RHEOSTAT_ADC_CHANNEL, 1, ADC_SampleTime_3Cycles);
92
93
94 // 使能 DMA 请求 after last transfer (multi-ADC mode)
95 ADC_MultiModeDMARequestAfterLastTransferCmd(ENABLE);
96 // 使能 ADC DMA
97 ADC_DMACmd(RHEOSTAT_ADC1, ENABLE);
98
99 // 使能 ADC
100 ADC_Cmd(RHEOSTAT_ADC1, ENABLE);
101 ADC_Cmd(RHEOSTAT_ADC2, ENABLE);
102 ADC_Cmd(RHEOSTAT_ADC3, ENABLE);
103
104 //开始 adc 转换, 软件触发
105 ADC_SoftwareStartConv(RHEOSTAT_ADC1);
106 // ADC_SoftwareStartConv(RHEOSTAT_ADC2);
107 // ADC_SoftwareStartConv(RHEOSTAT_ADC3);
108 }
```

首先, 我们使用了 `DMA_InitTypeDef` 定义了一个 DMA 初始化类型变量, 该结构体内容我们在 DMA 篇已经做了非常详细的讲解; 另外还使用 `ADC_InitTypeDef` 和 `ADC_CommonInitTypeDef` 结构体分别定义一个 ADC 初始化和 ADC 通用类型变量, 这两个结构体我们之前已经有详细讲解。

调用 `RCC_APB2PeriphClockCmd()` 开启 ADC 时钟以及 `RCC_AHB1PeriphClockCmd()` 开启 DMA 时钟。

我们需要对 DMA 进行必要的配置。首先设置外设基地址就是 ADC 的通用规则数据寄存器地址；存储器的地址就是我们指定的数据存储区空间，ADC_ConvertedValue 是我们定义的一个全局数组名，它是一个无符号 32 位有三个元素的整数数字；ADC 规则转换对应只有一个数据寄存器所以地址不能递增，我们指定的存储区也需要递增地址。ADC 的通用规则数据寄存器是 32 位有效，我们配置 ADC 为 DMA 模式 2，设置数据大小为字大小。ADC 配置为连续转换模式 DMA 也设置为循环传输模式。设置好 DMA 相关参数后就使能 DMA 的 ADC 通道。

接下来我们使用 ADC_CommonInitTypeDef 结构体变量 ADC_CommonInitStructure 来配置 ADC 为三重 ADC 交替模式、分频系数为 4、需要设置 DMA 模式 2、10 个周期的采样延迟，并调用 ADC_CommonInit 函数完成 ADC 通用工作环境配置。

我们使用 ADC_InitTypeDef 结构体变量 ADC_InitStructure 来配置 ADC1 为 12 位分辨率、不使用扫描模式、启动连续转换、使用内部软件触发无需外部触发事件、使用右对齐数据格式、转换通道为 1，并调用 ADC_Init 函数完成 ADC1 工作环境配置。ADC2 和 ADC3 使用与 ADC1 相同配置即可。

ADC_RegularChannelConfig 函数用来绑定 ADC 通道转换顺序和采样时间。绑定 ADC 通道引脚并设置相应的转换顺序。

ADC_MultiModeDMARequestAfterLastTransferCmd 函数控制是否使能 ADC 的 DMA 请求，如果使能请求，并调用 ADC_DMACmd 函数使能 DMA，则在 ADC 转换完成后就请求 DMA 实现数据传输。三重模式只需使能 ADC1 的 DMA 通道。

ADC_Cmd 函数控制 ADC 转换启动和停止。

最后，如果使用软件触发需要调用 ADC_SoftwareStartConvCmd 函数进行使能配置。

主函数

代码清单 31-14 主函数

```
1 #include "stm32f4xx.h"
2 #include "../usart/bsp_debug_usart.h"
3 #include "../adc/bsp_adc.h"
4
5
6 // ADC1 转换的电压值通过 MDA 方式传到 SRAM
7 extern __IO uint32_t ADC_ConvertedValue[3];
8
9 // 局部变量，用于保存转换计算后的电压值
10 float ADC_ConvertedValueLocal[3];
11
12 static void Delay(__IO uint32_t nCount) //简单的延时函数
13 {
14     for (; nCount != 0; nCount--);
15 }
16
17 /**
18  * @brief 主函数
19  * @param 无
20  * @retval 无
21  */
22 int main(void)
23 {
```

```
24  /*初始化 USART 配置模式为 115200 8-N-1, 中断接收*/
25  Debug_USART_Config();
26  Rheostat_Init();
27
28  while (1) {
29      Delay(0xffffee);
30
31      ADC_ConvertedValueLocal[0] =(float)((uint16_t)ADC_ConvertedValue[0]*3.3/4096);
32      ADC_ConvertedValueLocal[1] =(float)((uint16_t)ADC_ConvertedValue[1]*3.3/4096);
33      ADC_ConvertedValueLocal[2] =(float)((uint16_t)ADC_ConvertedValue[2]*3.3/4096);
34
35      printf("\r\n The current AD value = 0x%08X \r\n", ADC_ConvertedValue[0]);
36      printf("\r\n The current AD value = 0x%08X \r\n", ADC_ConvertedValue[1]);
37      printf("\r\n The current AD value = 0x%08X \r\n", ADC_ConvertedValue[2]);
38
39      printf("\r\n The current ADC1 value = %f V \r\n",ADC_ConvertedValueLocal[0]);
40      printf("\r\n The current ADC2 value = %f V \r\n",ADC_ConvertedValueLocal[1]);
41      printf("\r\n The current ADC3 value = %f V \r\n",ADC_ConvertedValueLocal[2]);
42  }
43 }
```

主函数先调用 `USARTx_Config` 函数配置调试串口相关参数，函数定义在 `bsp_debug_usart.c` 文件中。

接下来调用 `Rheostat_Init` 函数进行 ADC 初始化配置并启动 ADC。`Rheostat_Init` 函数是定义在 `bsp_adc.c` 文件中，它只是简单的分别调用 `Rheostat_ADC_GPIO_Config()` 和 `Rheostat_ADC_Mode_Config()`。

`Delay` 函数只是一个简单的延时函数。

我们配置了 DMA 数据传输所以它会自动把 ADC 转换完成后数据保存到数组变量 `ADC_ConvertedValue` 内，根据 DMA 模式 2 的数据存放规则，`ADC_ConvertedValue[0]` 的低 16 位存放 ADC1 数据、高 16 位存放 ADC2 数据，`ADC_ConvertedValue[1]` 的低 16 位存放 ADC3 数据、高 16 位存放 ADC1 数据，`ADC_ConvertedValue[2]` 的低 16 位存放 ADC2 数据、高 16 位存放 ADC3 数据，我们可以根据需要提取出对应 ADC 的转换结果数据。经过简单地计算就可以得到每个 ADC 对应的实际电压。

最后就是把相关数据打印至串口调试助手。

31.6.3 下载验证

保证开发板相关硬件连接正确，用 USB 线连接开发板“USB 转串口”接口跟电脑，在电脑端打开串口调试助手，把编译好的程序下载到开发板。在串口调试助手可看到不断有数据从开发板传输过来，此时我们旋转电位器改变其电阻值，那么对应的数据也会有变化。